

```

import bpy
import random
import math
import os
from mathutils import Vector

#
=====
=
# PART 1: SPINNING PLANETS & MOON SETUP
#
=====
=
planets_spin = {
    "Mercury": 0.8, "Venus": -0.5, "Earth": 1.5, "Mars": 1.4,
    "Jupiter": 3.5, "Saturn": 3.0, "Uranus": -2.0, "Neptune": 2.2, "Pluto": 0.7
}

for name, speed in planets_spin.items():
    obj = bpy.data.objects.get(name)
    if obj:
        obj.driver_remove("rotation_euler", 2)
        driver = obj.driver_add("rotation_euler", 2).driver
        driver.expression = f"(frame * {speed}) / 100"

moon = bpy.data.objects.get("Moon")
if moon:
    moon.driver_remove("rotation_euler", 2)
    moon_driver = moon.driver_add("rotation_euler", 2).driver
    moon_driver.expression = "(frame * 4.5) / 100"

#
=====
=
# PART 2: AUTOMATED PROCEDURAL TEXTURING SYSTEM WITH SHADOW BOOST
#
=====
=
texture_path = r"C:\Users\Janine Espares\Downloads\textures\photo-wall-texture-pattern.jpg"
mat_name = "Procedural_Asteroid_Material"
asteroid_mat = bpy.data.materials.get(mat_name) or bpy.data.materials.new(name=mat_name)

asteroid_mat.use_nodes = True

```

```
nodes = asteroid_mat.node_tree.nodes
links = asteroid_mat.node_tree.links
nodes.clear()
```

```
node_principled = nodes.new(type='ShaderNodeBsdfPrincipled')
node_output = nodes.new(type='ShaderNodeOutputMaterial')
node_texture = nodes.new(type='ShaderNodeTexImage')
node_coord = nodes.new(type='ShaderNodeTexCoord')
```

```
links.new(node_coord.outputs['Object'], node_texture.inputs['Vector'])
links.new(node_texture.outputs['Color'], node_principled.inputs['Base Color'])
links.new(node_principled.outputs['BSDF'], node_output.inputs['Surface'])
links.new(node_texture.outputs['Color'], node_principled.inputs['Emission Color'])
node_principled.inputs['Emission Strength'].default_value = 0.05
node_texture.projection = 'BOX'
node_texture.projection_blend = 0.2
```

```
if os.path.exists(texture_path):
    try: node_texture.image = bpy.data.images.load(texture_path)
    except: pass
```

```
#
=====
=
# PART 3: PERFECTLY SNUG ASTEROID BELT GENERATION
#
=====
=
NUM_ASTEROIDS = 300
BELT_INNER_RADIUS = 7.7
BELT_OUTER_RADIUS = 8.4
BELT_THICKNESS = 0.05
```

```
old_coll = bpy.data.collections.get("Asteroid Belt Pure")
if old_coll:
    for obj in old_coll.objects: bpy.data.objects.remove(obj, do_unlink=True)
    bpy.data.collections.remove(old_coll)
```

```
belt_coll = bpy.data.collections.new("Asteroid Belt Pure")
bpy.context.scene.collection.children.link(belt_coll)
```

```
verts = [(1,1,1), (1,-1,1), (-1,-1,1), (-1,1,1), (1,1,-1), (1,-1,-1), (-1,-1,-1), (-1,1,-1)]
faces = [(0,1,2,3), (4,7,6,5), (0,4,5,1), (1,5,6,2), (2,6,7,3), (4,0,3,7)]
```

```

mesh_data = bpy.data.meshes.new("Asteroid_Mesh")
mesh_data.from_pydata(verts, [], faces)
mesh_data.update()

for vert in mesh_data.vertices:
    vert.co.x += random.uniform(-0.3, 0.3)
    vert.co.y += random.uniform(-0.3, 0.3)
    vert.co.z += random.uniform(-0.3, 0.3)

base_rock = bpy.data.objects.new("Asteroid_Master", mesh_data)

for i in range(NUM_ASTEROIDS):
    rock_copy = base_rock.copy()
    rock_copy.data = mesh_data.copy()
    belt_coll.objects.link(rock_copy)
    rock_copy.data.materials.append(asteroid_mat)

    angle = random.uniform(0, 2 * math.pi)
    distance = random.uniform(BELT_INNER_RADIUS, BELT_OUTER_RADIUS)
    rock_copy.location = (math.cos(angle) * distance, math.sin(angle) * distance,
random.uniform(-BELT_THICKNESS, BELT_THICKNESS))

    sx, sy, sz = random.uniform(0.01, 0.035) , random.uniform(0.01, 0.035),
random.uniform(0.01, 0.035)
    rock_copy.scale = (sx, sy, sz)
    rock_copy.rotation_euler = (random.uniform(0, 6.2), random.uniform(0, 6.2),
random.uniform(0, 6.2))

    orbit_speed = random.uniform(0.4, 0.7) / (distance * 0.1)
    driver_x = rock_copy.driver_add("location", 0).driver
    driver_x.expression = f"cos((frame * {orbit_speed:4f}) / 100 + {angle:4f}) * {distance:4f}"
    driver_y = rock_copy.driver_add("location", 1).driver
    driver_y.expression = f"sin((frame * {orbit_speed:4f}) / 100 + {angle:4f}) * {distance:4f}"

#
=====
=
# PART 4: HIGH-REALISM SPACE SHUTTLE GENERATION (SMOOTH BLUEPRINT
Blueprints)
#
=====
=
from mathutils import Vector

```

```

for name in ["Space_Shuttle", "Shuttle_Flight_Path"]:
    old_obj = bpy.data.objects.get(name)
    if old_obj: bpy.data.objects.remove(old_obj, do_unlink=True)

# Delete old windows if re-running
for obj in bpy.data.objects:
    if "Shuttle_Window" in obj.name:
        bpy.data.objects.remove(obj, do_unlink=True)

# 1. Recreate Flight Path Curve
curve_data = bpy.data.curves.new('ShuttlePath', type='CURVE')
curve_data.dimensions = '3D'
shuttle_curve = bpy.data.objects.new('Shuttle_Flight_Path', curve_data)
bpy.context.scene.collection.objects.link(shuttle_curve)

points = [(-3.0, 3.0, 0.5), (0.0, 4.5, -0.5), (3.0, 3.0, 0.3), (4.5, 0.0, 0.8), (0.0, -4.5, -0.2)]
spline = curve_data.splines.new('BEZIER')
spline.bezier_points.add(len(points) - 1)
for idx, pt in enumerate(points):
    spline.bezier_points[idx].co = pt
    spline.bezier_points[idx].handle_left_type = 'AUTO'
    spline.bezier_points[idx].handle_right_type = 'AUTO'
spline.use_cyclic_u = True
curve_data.resolution_u = 64

# 2. Detailed Mesh Construction Map for an Authentic Orbiter Structure
shuttle_mesh = bpy.data.meshes.new("Realistic_Shuttle_Mesh")
shuttle_obj = bpy.data.objects.new("Space_Shuttle", shuttle_mesh)
bpy.context.scene.collection.objects.link(shuttle_obj)

# Explicit structural coordinate layouts matching the scaling ratios of the real orbiter
s_verts = [
    # ---- MAIN FUSELAGE CAPSULE CORE ----
    Vector((0.0, 1.6, -0.05)), # 0: Nose Tip Front
    Vector((0.15, 1.2, 0.15)), # 1: Nose Upper Right
    Vector((-0.15, 1.2, 0.15)), # 2: Nose Upper Left
    Vector((0.22, 1.2, -0.15)), # 3: Nose Lower Right
    Vector((-0.22, 1.2, -0.15)), # 4: Nose Lower Left

    Vector((0.32, 0.6, 0.28)), # 5: Cockpit Canopy Top Right
    Vector((-0.32, 0.6, 0.28)), # 6: Cockpit Canopy Top Left
    Vector((0.38, 0.6, -0.2)), # 7: Cabin Base Right
    Vector((-0.38, 0.6, -0.2)), # 8: Cabin Base Left

```

Vector((0.4, -0.4, 0.3)), # 9: Mid Fuselage Upper Right
Vector((-0.4, -0.4, 0.3)), # 10: Mid Fuselage Upper Left
Vector((0.42, -0.4, -0.25)), # 11: Mid Fuselage Lower Right
Vector((-0.42, -0.4, -0.25)), # 12: Mid Fuselage Lower Left

Vector((0.38, -1.4, 0.28)), # 13: Aft Fuselage Upper Right
Vector((-0.38, -1.4, 0.28)), # 14: Aft Fuselage Upper Left
Vector((0.38, -1.4, -0.22)), # 15: Aft Fuselage Lower Right
Vector((-0.38, -1.4, -0.22)), # 16: Aft Fuselage Lower Left

---- DELTA WINGS (Thick wing roots, tapered trailing edge sweeps) ----

Vector((1.75, -1.2, -0.12)), # 17: Starboard Wingtip Upper
Vector((1.75, -1.25, -0.22)), # 18: Starboard Wingtip Lower
Vector((-1.75, -1.2, -0.12)), # 19: Port Wingtip Upper
Vector((-1.75, -1.25, -0.22)), # 20: Port Wingtip Lower
Vector((0.42, 0.1, -0.1)), # 21: Starboard Leading Edge Root
Vector((-0.42, 0.1, -0.1)), # 22: Port Leading Edge Root

---- VERTICAL STABILIZER (Tail Fin Assembly) ----

Vector((0.0, -0.5, 0.32)), # 23: Tail Fin Forward Root
Vector((0.0, -1.4, 1.15)), # 24: Tail Fin Top Apex
Vector((0.0, -1.45, 0.28)), # 25: Tail Fin Trailing Base

---- OMS PODS & PROPULSION ENGINE BLOCK ----

Vector((0.25, -1.4, 0.42)), # 26: Starboard OMS Pod Upper Peak
Vector((-0.25, -1.4, 0.42)), # 27: Port OMS Pod Upper Peak
Vector((0.0, -1.6, 0.0)) # 28: Center Engine Thrust Exit Point

]

s_faces = [

Nose Section & Window Chamfers

(0, 1, 2), (0, 3, 1), (0, 2, 4), (0, 4, 3),

Cockpit Window Canopy Enclosure

(1, 5, 6, 2), (1, 3, 7, 5), (2, 6, 8, 4), (3, 4, 8, 7),

Mid-Body Cargo Bay Hull Panels

(5, 9, 10, 6), (5, 7, 11, 9), (6, 10, 12, 8), (7, 8, 12, 11),

Aft Structural Assembly

(9, 13, 14, 10), (9, 11, 15, 13), (10, 14, 16, 12), (11, 12, 16, 15),

Starboard Aerodynamic Delta Wing Plates

(21, 9, 13, 17), (21, 17, 18, 11), (11, 18, 15), (13, 15, 18, 17),

Port Aerodynamic Delta Wing Plates

(22, 19, 14, 10), (22, 12, 20, 19), (12, 16, 20), (14, 17, 20, 16),

```

# Vertical Tail Fin Extrusion Surfaces
(23, 24, 25), (13, 23, 25, 15), (14, 16, 25, 23),
# Engine Thruster Base Shell
(13, 26, 28, 15), (14, 16, 28, 27), (26, 27, 28), (13, 14, 27, 26)
]

shuttle_mesh.from_pydata(s_verts, [], s_faces)
shuttle_mesh.update()

# 3. Add High-Fidelity Geometry Modifiers
subsurf = shuttle_obj.modifiers.new(name="Smooth_Subdiv", type='SUBSURF')
subsurf.levels = 3
subsurf.render_levels = 3

edgesplit = shuttle_obj.modifiers.new(name="Edge_Sharpness", type='EDGE_SPLIT')
edgesplit.split_angle = 0.523599 # 30 degrees

for poly in shuttle_mesh.polygons:
    poly.use_smooth = True

shuttle_obj.scale = (0.16, 0.16, 0.16)

# 4. Pure White Shuttle Material
shuttle_mat = bpy.data.materials.get("Shuttle_Realistic_Material") or
bpy.data.materials.new(name="Shuttle_Realistic_Material")
shuttle_mat.use_nodes = True
s_nodes = shuttle_mat.node_tree.nodes
s_links = shuttle_mat.node_tree.links
s_nodes.clear()

node_output = s_nodes.new(type='ShaderNodeOutputMaterial')
node_principled = s_nodes.new(type='ShaderNodeBsdfPrincipled')

# Set shuttle to pure white
node_principled.inputs['Base Color'].default_value = (1.0, 1.0, 1.0, 1.0)
node_principled.inputs['Roughness'].default_value = 0.5

s_links.new(node_principled.outputs['BSDF'], node_output.inputs['Surface'])
shuttle_obj.data.materials.append(shuttle_mat)

# 5. Airplane-Style Passenger Windows (Black Dots)
window_mat = bpy.data.materials.get("Shuttle_Window_Material") or
bpy.data.materials.new(name="Shuttle_Window_Material")

```

```

window_mat.use_nodes = True
w_principled = window_mat.node_tree.nodes.get("Principled BSDF")
if w_principled:
    w_principled.inputs['Base Color'].default_value = (0.01, 0.01, 0.01, 1.0) # Dark black
    w_principled.inputs['Roughness'].default_value = 0.1 # Glossy glass

# Create a small base quad that will be subsurfed into a circular dot
win_mesh = bpy.data.meshes.new("Shuttle_Window_Mesh")
win_verts = [Vector((0, 0.025, 0.025)), Vector((0, -0.025, 0.025)), Vector((0, -0.025, -0.025)),
Vector((0, 0.025, -0.025))]
win_mesh.from_pydata(win_verts, [], [(0, 1, 2, 3)])
win_mesh.update()

num_windows = 8
y_start = 0.45 # Near cockpit
y_end = -0.15 # Towards aft section

for side in [1, -1]: # 1 = Starboard (Right), -1 = Port (Left)
    for i in range(num_windows):
        win_obj = bpy.data.objects.new(f"Shuttle_Window_{'Right' if side == 1 else 'Left'}_{i}",
win_mesh)
        bpy.context.scene.collection.objects.link(win_obj)
        win_obj.data.materials.append(window_mat)

        # Position interpolation along the fuselage
        y_pos = y_start + (y_end - y_start) * (i / max(1, num_windows - 1))
        x_pos = 0.42 * side # Pulled slightly outward to avoid subsurf clipping
        z_pos = 0.18 # Mid-height on the hull

        win_obj.location = (x_pos, y_pos, z_pos)

        # Make the square a perfect circular dot
        win_subsurf = win_obj.modifiers.new(name="Dot_Smooth", type='SUBSURF')
        win_subsurf.levels = 2

        # Lock to shuttle
        win_obj.parent = shuttle_obj

# 6. Attach path animation tracking data
constraint = shuttle_obj.constraints.new(type='FOLLOW_PATH')
constraint.target = shuttle_curve
constraint.use_fixed_location = False
constraint.use_curve_follow = True
constraint.forward_axis = 'FORWARD_Y'

```

```

constraint.up_axis = 'UP_Z'

shuttle_curve.data.use_path = True
shuttle_curve.data.path_duration = 200
shuttle_curve.data.driver_remove("eval_time")
path_driver = shuttle_curve.data.driver_add("eval_time").driver
path_driver.expression = "frame"

#
=====
=
# PART 5: MODIFIED CINEMATIC VLOG CAMERA SYSTEM
#
=====
=
for name in ["Cinematic_Tour_Camera", "Camera_Target_Focus"]:
    old_obj = bpy.data.objects.get(name)
    if old_obj: bpy.data.objects.remove(old_obj, do_unlink=True)

target_empty = bpy.data.objects.new("Camera_Target_Focus", None)
bpy.context.scene.collection.objects.link(target_empty)

cam_data = bpy.data.cameras.new("Cinematic_Tour_CamData")
tour_cam = bpy.data.objects.new("Cinematic_Tour_Camera", cam_data)
bpy.context.scene.collection.objects.link(tour_cam)
bpy.context.scene.camera = tour_cam

# Camera Tracking Constraints
track_constraint = tour_cam.constraints.new(type='TRACK_TO')
track_constraint.target = target_empty
track_constraint.track_axis = 'TRACK_NEGATIVE_Z'
track_constraint.up_axis = 'UP_Y'

fps = bpy.context.scene.render.fps
stay_dur = int(fps * 5.0)    # 5 seconds standard showcase duration
trans_dur = int(fps * 2.5)  # 2.5 seconds smooth transition time

tour_sequence = [
    "Sun", "Mercury", "Venus", "Earth", "Mars",
    "Asteroid_Belt", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto"
]

cinematic_settings = {
    "Sun": {"lens": 32},

```



```

"Mercury": {"dist": 4.5, "z_offset": 0.8, "lens": 35},
"Venus": {"dist": 7.0, "z_offset": 1.6, "lens": 35},
"Earth": {"dist": 8.5, "z_offset": 1.8, "lens": 28},
"Mars": {"dist": 6.0, "z_offset": 1.2, "lens": 35},
"Asteroid_Belt": {"lens": 28},
"Jupiter": {"dist": 5.5, "z_offset": 1.5, "lens": 35},
"Saturn": {"dist": 7.5, "z_offset": 1.2, "lens": 35},
"Uranus": {"dist": 5.5, "z_offset": 1.5, "lens": 35},
"Neptune": {"dist": 5.5, "z_offset": 1.5, "lens": 35},
"Pluto": {"dist": 5.0, "z_offset": 1.0, "lens": 35},
}

```

```

sun_obj = bpy.data.objects.get("Sun")
sun_center = sun_obj.location if sun_obj else Vector((0, 0, 0))
sun_scale = max(sun_obj.scale) if (sun_obj and sun_obj.scale) else 1.0

```

```

tour_cam.animation_data_clear()
target_empty.animation_data_clear()
cam_data.animation_data_clear()

```

```

def get_cinematic_vectors(p_name, frame, settings):
    p_obj = bpy.data.objects.get(p_name)
    if not p_obj: return Vector((0,0,0)), Vector((0,0,0))
    p_scale = max(p_obj.scale)

```

```

    bpy.context.scene.frame_set(int(frame))
    pos = p_obj.matrix_world.to_translation()

```

```

    bpy.context.scene.frame_set(int(frame) - 2)
    pos_prev = p_obj.matrix_world.to_translation()

```

```

    vel = (pos - pos_prev)
    if vel.length < 0.0001: vel = Vector((-pos.y, pos.x, 0))
    vel = vel.normalized()

```

```

    to_sun = (sun_center - pos).normalized()
    if to_sun.length < 0.0001: to_sun = Vector((0, -1, 0))

```

```

    if p_name == "Venus":
        cam_dir = (-to_sun * 1.2 - vel * 1.5).normalized()
    else:
        cam_dir = (to_sun * 1.2 - vel * 1.0).normalized()

```

```

    cam_pos = pos + cam_dir * (settings["dist"] * p_scale) + Vector((0, 0, settings["z_offset"] *
p_scale))
    return pos, cam_pos

```

```

# --- PRE-EARTH TIMELINE ALLOCATION ---

```

```

frames_allocated = {
    "Sun_Start": 1,
    "Sun_Pullback": 70,
    "Sun_End": 120,
    "Mercury_Arrive": 145,
    "Mercury_End": 195,
    "Venus_Arrive": 215,
    "Venus_End": 254,
    "Earth_Arrive": 314,
    "Earth_End": 314 + stay_dur,
}

```

```

current_frame = 1

```

```

for t_name in tour_sequence:

```

```

    p_obj = bpy.data.objects.get(t_name) if t_name != "Asteroid_Belt" else True
    if not p_obj: continue

```

```

    settings = cinematic_settings.get(t_name, {})
    lens_val = settings.get("lens", 35)

```

```

    if t_name == "Sun":

```

```

        current_frame = frames_allocated["Sun_Start"]
        bpy.context.scene.frame_set(current_frame)
        target_empty.location = sun_center
        tour_cam.location = sun_center + Vector((0, -sun_scale * 1.5, sun_scale * 0.15))
        cam_data.lens = lens_val
        target_empty.keyframe_insert(data_path="location", frame=current_frame)
        tour_cam.keyframe_insert(data_path="location", frame=current_frame)
        cam_data.keyframe_insert(data_path="lens", frame=current_frame)

```

```

        current_frame = frames_allocated["Sun_Pullback"]
        bpy.context.scene.frame_set(current_frame)
        tour_cam.location = sun_center + Vector((0, -sun_scale * 4.8, sun_scale * 0.6))
        target_empty.keyframe_insert(data_path="location", frame=current_frame)
        tour_cam.keyframe_insert(data_path="location", frame=current_frame)

```

```

        current_frame = frames_allocated["Sun_End"]
        bpy.context.scene.frame_set(current_frame)

```

```
tour_cam.location = sun_center + Vector((sun_scale * 0.4, -sun_scale * 5.2, sun_scale * 0.9))
```

```
target_empty.keyframe_insert(data_path="location", frame=current_frame)
```

```
tour_cam.keyframe_insert(data_path="location", frame=current_frame)
```

```
cam_data.keyframe_insert(data_path="lens", frame=current_frame)
```

```
elif t_name in ["Mercury", "Venus"]:
```

```
start_key = f"{t_name}_Arrive"
```

```
end_key = f"{t_name}_End"
```

```
arrive_f = frames_allocated[start_key]
```

```
end_f = frames_allocated[end_key]
```

```
for f in range(arrive_f, end_f + 1, 2):
```

```
    t_loc, c_loc = get_cinematic_vectors(t_name, f, settings)
```

```
    bpy.context.scene.frame_set(f)
```

```
    target_empty.location = t_loc
```

```
    tour_cam.location = c_loc
```

```
    cam_data.lens = lens_val
```

```
    target_empty.keyframe_insert(data_path="location", frame=f)
```

```
    tour_cam.keyframe_insert(data_path="location", frame=f)
```

```
    cam_data.keyframe_insert(data_path="lens", frame=f)
```

```
current_frame = end_f
```

```
elif t_name == "Earth":
```

```
    arrive_f = frames_allocated["Earth_Arrive"]
```

```
    end_f = frames_allocated["Earth_End"]
```

```
for f in range(arrive_f, end_f + 1, 2):
```

```
    t_loc, c_loc = get_cinematic_vectors(t_name, f, settings)
```

```
    bpy.context.scene.frame_set(f)
```

```
    target_empty.location = t_loc
```

```
    tour_cam.location = c_loc
```

```
    cam_data.lens = lens_val
```

```
    target_empty.keyframe_insert(data_path="location", frame=f)
```

```
    tour_cam.keyframe_insert(data_path="location", frame=f)
```

```
    cam_data.keyframe_insert(data_path="lens", frame=f)
```

```
current_frame = end_f
```

```
elif t_name == "Mars":
```

```
    current_frame += trans_dur
```

```
    arrive_f = current_frame
```

```
    end_f = arrive_f + stay_dur
```

```
for f in range(arrive_f, end_f + 1, 2):
```

```

t_loc, c_loc = get_cinematic_vectors(t_name, f, settings)
bpy.context.scene.frame_set(f)
target_empty.location = t_loc
tour_cam.location = c_loc
cam_data.lens = lens_val
target_empty.keyframe_insert(data_path="location", frame=f)
tour_cam.keyframe_insert(data_path="location", frame=f)
cam_data.keyframe_insert(data_path="lens", frame=f)
current_frame = end_f

```

```

elif t_name == "Asteroid Belt":
    current_frame += trans_dur
    bpy.context.scene.frame_set(current_frame)
    target_empty.location = Vector((8.0, 0.0, 0.0))
    tour_cam.location = Vector((6.4, -2.8, 0.7))
    cam_data.lens = lens_val
    target_empty.keyframe_insert(data_path="location", frame=current_frame)
    tour_cam.keyframe_insert(data_path="location", frame=current_frame)
    cam_data.keyframe_insert(data_path="lens", frame=current_frame)

```

```

    current_frame += stay_dur
    bpy.context.scene.frame_set(current_frame)
    target_empty.location = Vector((8.2, 0.2, -0.1))
    tour_cam.location = Vector((7.2, -1.5, 0.4))
    target_empty.keyframe_insert(data_path="location", frame=current_frame)
    tour_cam.keyframe_insert(data_path="location", frame=current_frame)
    cam_data.keyframe_insert(data_path="lens", frame=current_frame)

```

```

else:
    current_frame += trans_dur
    arrive_f = current_frame
    end_f = arrive_f + stay_dur

```

```

for f in range(arrive_f, end_f + 1, 2):
    t_loc, c_loc = get_cinematic_vectors(t_name, f, settings)
    bpy.context.scene.frame_set(f)
    target_empty.location = t_loc
    tour_cam.location = c_loc
    cam_data.lens = lens_val
    target_empty.keyframe_insert(data_path="location", frame=f)
    tour_cam.keyframe_insert(data_path="location", frame=f)
    cam_data.keyframe_insert(data_path="lens", frame=f)
current_frame = end_f

```

```

#
=====
=
# PART 6: SHUTTLE FINALE (HIGH-ANGLE OVERHEAD HERO SHOT)
#
=====
=
shuttle_obj = bpy.data.objects.get("Space_Shuttle")
if not shuttle_obj:
    for obj in bpy.data.objects:
        if "shuttle" in obj.name.lower():
            shuttle_obj = obj
            break

if shuttle_obj:
    transition_duration = int(fps * 2.5) # 2.5-second smooth pan to shuttle
    shuttle_follow_dur = int(fps * 10.0) # Clean, locked 10-second tracking window

    transition_start = current_frame
    transition_end = transition_start + transition_duration
    hero_end = transition_end + shuttle_follow_dur

    hero_offset = Vector((-1.5, -3.0, 4.0))

    # --- Phase 1: Pluto to Shuttle High-Angle Transition ---
    bpy.context.scene.frame_set(transition_start)
    shuttle_loc = shuttle_obj.matrix_world.to_translation()
    target_empty.location = shuttle_loc

    target_empty.keyframe_insert(data_path="location", frame=transition_start)
    tour_cam.keyframe_insert(data_path="location", frame=transition_start)
    cam_data.keyframe_insert(data_path="lens", frame=transition_start)

    bpy.context.scene.frame_set(transition_end)
    shuttle_loc = shuttle_obj.matrix_world.to_translation()
    target_empty.location = shuttle_loc
    tour_cam.location = shuttle_loc + hero_offset
    cam_data.lens = 45 # Tightened framing zoom

    target_empty.keyframe_insert(data_path="location", frame=transition_end)
    tour_cam.keyframe_insert(data_path="location", frame=transition_end)
    cam_data.keyframe_insert(data_path="lens", frame=transition_end)

```

```

# --- Phase 2: Complete 10-Second Semi-Eagle Eye Tracking Engine ---
for f in range(transition_end + 2, hero_end + 1, 2):
    bpy.context.scene.frame_set(f)
    s_pos_curr = shuttle_obj.matrix_world.to_translation()
    target_empty.location = s_pos_curr
    tour_cam.location = s_pos_curr + hero_offset

    target_empty.keyframe_insert(data_path="location", frame=f)
    tour_cam.keyframe_insert(data_path="location", frame=f)

current_frame = hero_end

#
=====
=
# PART 7: SINGLE GRAND SOLAR SYSTEM REVEAL (GLITCH-FREE FIXED LERP)
#
=====
=
reveal_duration = int(fps * 8.0) # 8-second smooth continuous pullback
reveal_start = current_frame
reveal_end = reveal_start + reveal_duration

bpy.context.scene.frame_set(reveal_start)
start_cam_loc = tour_cam.location.copy()
start_target_loc = target_empty.location.copy()
end_cam_loc = sun_center + Vector((0, -45, 18))
start_lens = cam_data.lens
end_lens = 22

for f in range(reveal_start, reveal_end + 1, 2):
    t = (f - reveal_start) / reveal_duration
    bpy.context.scene.frame_set(f)

    tour_cam.location = start_cam_loc.lerp(end_cam_loc, t)
    target_empty.location = start_target_loc.lerp(sun_center, t)
    cam_data.lens = start_lens + (end_lens - start_lens) * t

    tour_cam.keyframe_insert(data_path="location", frame=f)
    target_empty.keyframe_insert(data_path="location", frame=f)
    cam_data.keyframe_insert(data_path="lens", frame=f)

current_frame = reveal_end

```

```
bpy.context.scene.frame_set(1)
bpy.context.scene.frame_end = current_frame
```

```
# Global Bezier Spline Interp System
```

```
def apply_smooth_bezier(obj):
```

```
    if obj.animation_data and obj.animation_data.action:
```

```
        action = obj.animation_data.action
```

```
        fcurves = []
```

```
        if hasattr(action, "fcurves"):
```

```
            fcurves = action.fcurves
```

```
        elif hasattr(action, "layers"):
```

```
            for layer in action.layers:
```

```
                if hasattr(layer, "channels"):
```

```
                    for channel in layer.channels:
```

```
                        if hasattr(channel, "fcurves"):
```

```
                            fcurves.extend(channel.fcurves)
```

```
        for fcurve in fcurves:
```

```
            for kp in fcurve.keyframe_points:
```

```
                if kp.frame >= transition_end:
```

```
                    kp.interpolation = 'LINEAR'
```

```
                else:
```

```
                    kp.interpolation = 'BEZIER'
```

```
                    kp.handle_left_type = 'AUTO'
```

```
                    kp.handle_right_type = 'AUTO'
```

```
for item in [tour_cam, target_empty, cam_data]:
```

```
    apply_smooth_bezier(item)
```

```
#
```

```
=====
```

```
=
```

```
# PART 8: TARGETED REGIONAL METEOR SYSTEM (ORIENTED HORIZONTALLY)
```

```
#
```

```
=====
```

```
=
```

```
old_meteor_coll = bpy.data.collections.get("Meteor Shower Pure")
```

```
if old_meteor_coll:
```

```
    for obj in old_meteor_coll.objects: bpy.data.objects.remove(obj, do_unlink=True)
```

```
    bpy.data.collections.remove(old_meteor_coll)
```

```
meteor_coll = bpy.data.collections.new("Meteor Shower Pure")
```

```
bpy.context.scene.collection.children.link(meteor_coll)
```

```

meteor_mat = bpy.data.materials.get("Meteor_Atmospheric_Material") or
bpy.data.materials.new(name="Meteor_Atmospheric_Material")
meteor_mat.use_nodes = True
m_nodes = meteor_mat.node_tree.nodes
m_links = meteor_mat.node_tree.links
m_nodes.clear()

m_principled = m_nodes.new(type='ShaderNodeBsdfPrincipled')
m_output = m_nodes.new(type='ShaderNodeOutputMaterial')
m_coord = m_nodes.new(type='ShaderNodeTexCoord')
m_separate = m_nodes.new(type='ShaderNodeSeparateXYZ')
m_ramp = m_nodes.new(type='ShaderNodeValToRGB')
m_obj_info = m_nodes.new(type='ShaderNodeObjectInfo')
m_math_brightness = m_nodes.new(type='ShaderNodeMath')

m_links.new(m_coord.outputs['Generated'], m_separate.inputs['Vector'])
m_links.new(m_separate.outputs['Y'], m_ramp.inputs['Fac'])

m_ramp.color_ramp.elements[0].position = 0.0
m_ramp.color_ramp.elements[0].color = (0.05, 0.005, 0.0, 1.0)
m_mid = m_ramp.color_ramp.elements.new(0.7)
m_mid.color = (1.0, 0.25, 0.01, 1.0)
m_ramp.color_ramp.elements[-1].position = 1.0
m_ramp.color_ramp.elements[-1].color = (1.0, 0.95, 0.88, 1.0)

m_math_brightness.operation = 'MULTIPLY'
m_math_brightness.inputs[1].default_value = 16.0
m_links.new(m_obj_info.outputs['Random'], m_math_brightness.inputs[0])

m_links.new(m_ramp.outputs['Color'], m_principled.inputs['Base Color'])
m_links.new(m_ramp.outputs['Color'], m_principled.inputs['Emission Color'])
m_links.new(m_math_brightness.outputs['Value'], m_principled.inputs['Emission Strength'])
m_links.new(m_principled.outputs['BSDF'], m_output.inputs['Surface'])

m_mesh = bpy.data.meshes.new("Meteor_Base_Mesh")
m_verts = [
    Vector((0, 0.1, 0)),
    Vector((0.015, 0, 0.015)),
    Vector((-0.015, 0, 0.015)),
    Vector((-0.015, 0, -0.015)),
    Vector((0.015, 0, -0.015)),
    Vector((0, -2.5, 0))
]

```



```

m_faces = [(0,1,2), (0,2,3), (0,3,4), (0,4,1), (5,2,1), (5,3,2), (5,4,3), (5,1,4)]
m_mesh.from_pydata(m_verts, [], m_faces)
m_mesh.update()
base_meteor_obj = bpy.data.objects.new("Meteor_Master", m_mesh)

# --- Region A: Atmospheric Entry ---
for planet_name in ["Earth", "Mars"]:
    planet_obj = bpy.data.objects.get(planet_name)
    if not planet_obj: continue

# MINIMIZED METEORS: Reduced range from 8 to 2 for less clutter
for j in range(2):
    met_streak = base_meteor_obj.copy()
    met_streak.data = m_mesh.copy()
    meteor_coll.objects.link(met_streak)
    met_streak.data.materials.append(meteor_mat)
    met_streak.parent = planet_obj

    spawn_angle = random.uniform(0, 2 * math.pi)
    local_radius = random.uniform(1.2, 1.6)

    start_x = math.cos(spawn_angle) * local_radius
    start_y = math.sin(spawn_angle) * local_radius
    start_z = random.uniform(-0.1, 0.1)

    entry_lifespan = random.randint(35, 60)
    vel_x = (-math.sin(spawn_angle) * 0.5 - start_x * 0.2) / entry_lifespan
    vel_y = (math.cos(spawn_angle) * 0.5 - start_y * 0.2) / entry_lifespan
    vel_z = random.uniform(-0.005, 0.005)

    direction = Vector((vel_x, vel_y, vel_z)).normalized()
    met_streak.rotation_mode = 'QUATERNION'
    met_streak.rotation_quaternion = Vector((0, 1, 0)).rotation_difference(direction)

    timeline_offset = random.randint(0, 200)
    for axis_idx, vel_val, start_val in [(0, vel_x, start_x), (1, vel_y, start_y), (2, vel_z, start_z)]:
        drv = met_streak.driver_add("location", axis_idx).driver
        drv.expression = f"{start_val:.4f} + ((frame + {timeline_offset}) % {entry_lifespan}) * {vel_val:.4f}"

    sf = random.uniform(0.12, 0.35)
    met_streak.scale = (sf, sf, sf * random.uniform(0.6, 1.8))

# --- Region B: Asteroid Belt Clusters ---

```

```

NUM_BELT_METEORS = 25
for k in range(NUM_BELT_METEORS):
    belt_met = base_meteor_obj.copy()
    belt_met.data = m_mesh.copy()
    meteor_coll.objects.link(belt_met)
    belt_met.data.materials.append(meteor_mat)

    b_angle = random.uniform(0, 2 * math.pi)
    b_distance = random.uniform(BELT_INNER_RADIUS, BELT_OUTER_RADIUS)
    b_speed = random.uniform(1.2, 2.2) / (b_distance * 0.1)

    drv_x = belt_met.driver_add("location", 0).driver
    drv_x.expression = f"cos((frame * {b_speed:.4f}) / 100 + {b_angle:.4f}) * {b_distance:.4f}"

    drv_y = belt_met.driver_add("location", 1).driver
    drv_y.expression = f"sin((frame * {b_speed:.4f}) / 100 + {b_angle:.4f}) * {b_distance:.4f}"

    belt_met.location.z = random.uniform(-BELT_THICKNESS, BELT_THICKNESS)

    drv_rot = belt_met.driver_add("rotation_euler", 2).driver
    drv_rot.expression = f"((frame * {b_speed:.4f}) / 100 + {b_angle:.4f})"

    s_factor = random.uniform(0.15, 0.45)
    belt_met.scale = (s_factor, s_factor, s_factor * random.uniform(1.0, 3.0))

print("[Success] High-angle tracking setup, duplicate reveal fixed, 3D shuttle body initialized,
and planetary meteors minimized.")

```